



Applied Cryptography

Summer 2026

Part 2: Real-World Cryptography

2.2: The Story of RC4

Nadim Kobeissi

<https://appliedcryptography.page>

Section 1

About RC4

The Rise and Fall of RC4

- A cryptographic tale of triumph and tragedy
 - **Birth (1987):** Proprietary stream cipher at RSA Security
 - **Rise (1990s):** Became the world's most widely deployed stream cipher
 - **Golden Era (1990s-2000s):** Powered WEP, SSL, and TLS protocols globally
 - **Decline (2001-2015):** A series of devastating cryptanalytic breakthroughs
 - **Demise (2015):** Formal prohibition by IETF, browser vendors
- This story teaches us about cryptographic lifecycle management, the importance of formal security analysis, and the challenges of maintaining backward compatibility

What is RC4?

- **RC4** (Rivest Cipher 4) is a stream cipher designed for speed and simplicity
 - **Stream cipher:** Encrypts data one byte at a time
 - **Variable key length:** Supports keys from 1 to 256 bytes
 - **Symmetric encryption:** Same key used for encryption and decryption
- Key characteristics that made it popular:
 - Extremely fast in software (no complex operations)
 - Small memory footprint (only 256 bytes of state)
 - Simple implementation (just a few lines of code)
- Used extensively in protocols like WEP (Wi-Fi), SSL/TLS, and SSH

Origins of RC4

- **1987:** Ron Rivest designs RC4 at RSA Security
 - Originally a trade secret, not published
 - “RC” stands for “Ron’s Code” or “Rivest Cipher”
 - Designed for practical applications requiring fast encryption
- **1994:** Algorithm anonymously posted to Cypherpunks mailing list
 - Source code leaked, breaking RSA’s trade secret
 - Called “Alleged RC4” or “ARCFOUR” due to legal concerns
 - Quickly reverse-engineered and verified as authentic
- **Impact of the leak:**
 - Made RC4 freely available to implementers worldwide
 - Led to its widespread adoption in internet protocols
 - Ironically helped RSA by making their cipher ubiquitous

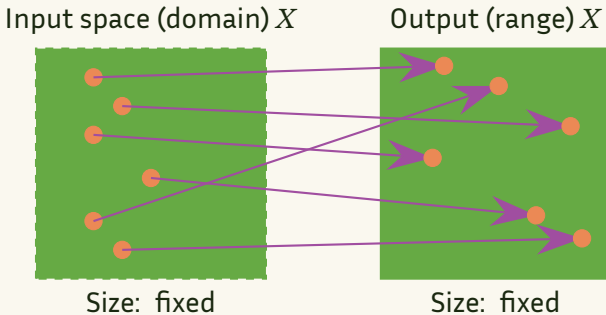
RC4's Key Scheduling Algorithm (KSA)

- **Step 1:** Initialize the state array S with values 0 through 255
 - for $i = 0$ to 255: $S[i] = i$
- **Step 2:** Scramble the state array using the key
 - $j = 0$
 - for $i = 0$ to 255:
 - $j = (j + S[i] + \text{key}[i \bmod \text{keylength}]) \bmod 256$
 - $\text{swap}(S[i], S[j])$
- **Purpose:** Create a pseudo-random permutation of 0-255 based on the key
- The quality of this initial scrambling is crucial for security

$$\text{PRP} : F_k : X \rightarrow X$$

Reminder

- **Bijective** (two-way)
 - **Injective**: no two inputs map to same output (no collisions)
 - **Surjective**: Every output has one corresponding input
- “Randomized”
- Relations between inputs not reflected in outputs



Limitations of One-Time Pad

Reminder

The Key Length Problem

One-time pad is not a particularly useful encryption scheme in practice.

- The key must be as long as the plaintext!
- This creates a chicken-and-egg situation:
 - To privately send n bits of information,
 - We must already privately share n bits of information.
- Impractical for most real-world applications.
 - Clearly this is not what we're doing when we use HTTPS,
 - or WhatsApp, or pay for something via a debit card...
- We need encryption schemes where the key can be smaller than the message.

Idea: find a way to expand the key

Reminder

- Given a key k_s of size $|k_s| < |m|$, find a way to obtain $|k_e| \geq |m|$
- In the real world, we have two kinds of symmetric encryption schemes:
 - **Block ciphers:** AES, 3DES, etc.
 - **Stream ciphers:** ChaCha20, RC4, etc.
- This is exactly what stream ciphers do!
 - Start with a small key k_s of a fixed size $|k_s| = \lambda$,
 - Magically expand it to k_e where $|k_e| \geq |m|$,
 - $\text{ENC}(K, M) = K \oplus M$

RC4's Pseudo-Random Generation Algorithm (PRGA)

- **Initialization:** Set counters $i = 0$, $j = 0$
- **For each byte to encrypt:**
 - $i = (i + 1) \bmod 256$
 - $j = (j + S[i]) \bmod 256$
 - $\text{swap}(S[i], S[j])$
 - $K = S[(S[i] + S[j]) \bmod 256] \leftarrow \text{keystream byte}$
 - $\text{ciphertext} = \text{plaintext} \text{ XOR } K$
- **Key insight:** The state array S is continuously modified
 - Each keystream byte depends on the entire previous history
 - Creates a very long period before repetition
- **Decryption:** Identical process (XOR is self-inverse)

RC4's golden age (1990s-2000s)

- **The most widely deployed stream cipher in history**
 - Estimated to secure over 50% of all SSL/TLS connections at its peak
 - Billions of devices worldwide relied on RC4 for encryption
- **Ubiquitous protocol adoption:**
 - **WEP (1997):** Wi-Fi security standard used RC4 exclusively
 - **SSL 3.0/TLS 1.0 (1996-1999):** RC4 as preferred cipher suite
 - **SSH-1 (1995):** Early secure shell implementations
 - **Microsoft Office:** Document password protection
 - **Adobe PDF:** File encryption standard
- **Why RC4 dominated:**
 - 5-10x faster than DES in software implementations
 - Minimal memory requirements (perfect for embedded systems)
 - Fewer export restrictions than strong block ciphers
 - Simple to implement correctly

Section 2

RC4 in WEP

Early RC4 weaknesses (1995-2000)

- **1995:** Roos discovered correlations between key bytes and initial permutation
 - Found that RC4's key scheduling leaks information about the key
 - Wagner and Goldberg identified flaws in SSL 3.0's use of RC4
 - Seemed like minor curiosities at the time...
- **1997:** Golić discovered more biases in the keystream
 - Certain byte positions showed statistical irregularities
 - Still considered mostly theoretical concerns
- **2000:** Jenkins found patterns in RC4's internal state
 - Identified correlations between consecutive keystream bytes
 - Cryptographic community began to worry about RC4's security
- **The stage was set** for a more devastating attack...

Weak keys in KSA

- **The Discovery:** Certain key patterns create predictable initial states^a
- **Weak Key Pattern:** Keys of the form $(K_1, K_2, \dots, K_n, 3, 255, \dots)$
 - When byte 3 of the key is 3, and byte 4 is 255
 - The KSA creates a predictable correlation in the state array
- **The Mathematics:**
 - After KSA completes, $S[1] = 3$ with high probability
 - In the first step of PRGA: $i = 1, j = S[1] = 3$
 - First keystream byte: $S[S[1] + S[3]] = S[3 + S[3]]$
 - This reveals information about the key!
- **Frequency:** About 1 in 256 keys exhibit this weakness

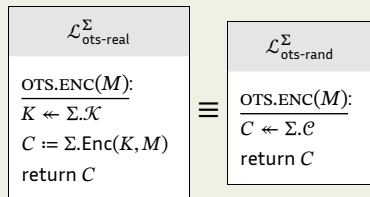
^a<https://appliedcryptography.page/paper/#rc4-ksa>

One-time secrecy of a SKE

Reminder

One-time Secrecy for SKE

An SKE scheme Σ has one-time secrecy if the following libraries are interchangeable:



An encryption scheme has one-time secrecy if its ciphertexts are uniformly distributed, when keys are sampled uniformly, kept secret, and used for only one encryption, and no matter how the plaintexts are chosen.

Does this apply to RC4, especially after the attacks?

Attacks mean distinguishability

RC4 getting less and less “indistinguishable from random”

RC4ENCRYPT(M):

$k \leftarrow \{\mathbf{0}, \mathbf{1}\}^\lambda$

$K := \text{RC4}(k, |M|)$

$C := K \oplus M$

return C

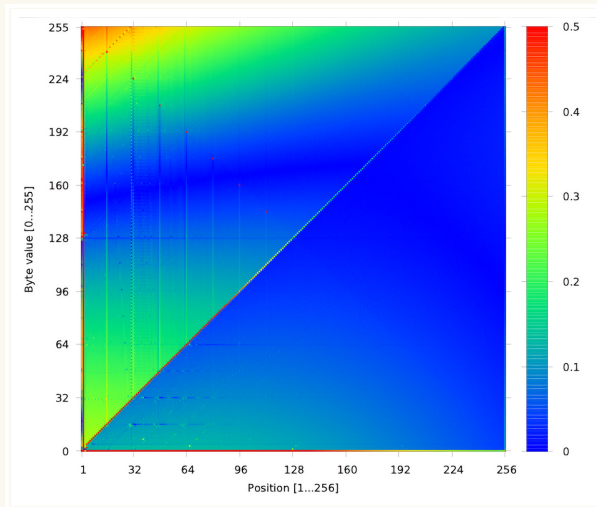
$\approx$

RANDOM(M):

$C \leftarrow \{\mathbf{0}, \mathbf{1}\}^{|M|}$

return C

In fact...



Source: Kenny Paterson

Attacks: from theory to practice

- **Step 1:** Collect many ciphertexts encrypted with related keys
 - Keys that differ only in the first few bytes
 - Wait for keys with the vulnerable pattern to appear
- **Step 2:** Identify weak keys from keystream patterns
 - Look for the telltale signatures in the first keystream byte
 - Discard non-vulnerable encryptions
- **Step 3:** Recover key bytes incrementally
 - Use the correlation to determine likely values for key bytes
 - Build up the key one byte at a time
- **Success rate:** With enough samples, key recovery becomes highly reliable

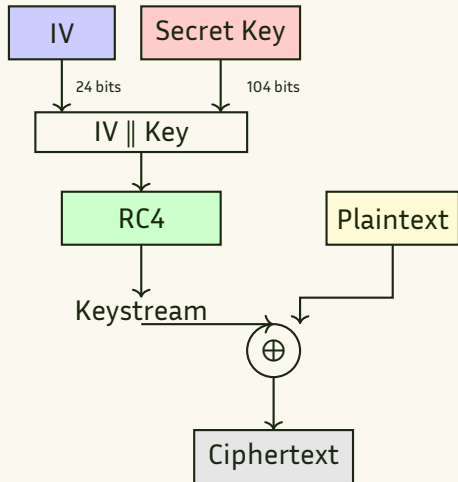
How WEP encryption works

- **Components:**

- 24-bit IV (Initialization Vector)
- 104-bit PSK (pre-shared key)
 - Derived from Wi-Fi password
 - Was limited to 40 bits at first due to US export controls!

- **Process:**

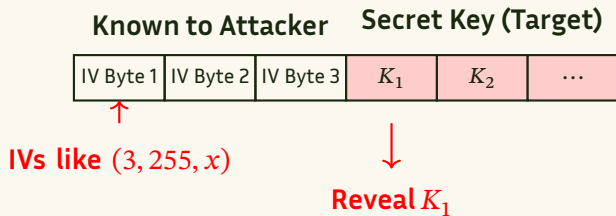
- Choose random IV for each packet
- Concatenate: IV || Secret Key
- Feed to RC4 to generate keystream
- XOR plaintext with keystream



WEP: the perfect storm

- **WEP (Wired Equivalent Privacy):** Wi-Fi security protocol from 1997
 - Used RC4 for encryption
 - 104-bit secret key shared among all devices
- **Vulnerable design choice:** Initialization Vectors (IVs)
 - To avoid key reuse, WEP prepends a 24-bit IV to each packet
 - RC4 key becomes: $(IV_1, IV_2, IV_3, K_1, K_2, \dots, K_n)$
 - IV is sent *in plaintext* with each packet!
- **Break:** This creates exactly the conditions the FMS (Fluhrer, Mantin, Shamir) attack exploited
 - Many related keys (same secret key, different IVs)
 - Attacker knows the first 3 bytes of each key (the IV)
 - Perfect setup for the FMS attack!

WEP's IV structure



- **The FMS Attack:** Exploits weak IV patterns of form $(A + 3, 255, x)$
- **Pattern:** To attack key byte K_A , wait for IVs $(A + 3, 255, x)$
- **Frequency:** About 1 in 65,536 packets match the vulnerable pattern
- **Collection:** Need multiple matching IVs per key byte for statistical attack
- **Result:** With enough packets, incrementally recover the entire key

Practical tools

- **AirSnort (2001):** First practical WEP cracking tool
 - Implemented the FMS attack
 - Could crack WEP keys with 1-5 million packets
 - Made WEP cracking accessible to non-experts
- **WEPCrack (2001):** Another early implementation
 - Demonstrated the attack's effectiveness
 - Confirmed that WEP was fundamentally broken
- **Aircrack-ng (2006):** Improved and optimized tools
 - Reduced required packets to hundreds of thousands
 - Added packet injection to speed up attacks
 - Still widely used today for security testing

Attack: step by step

- **Step 1:** Attacker sets Wi-Fi card to monitor mode
 - Passively collects all WEP-encrypted packets
 - No need to associate with the target network
- **Step 2:** Filter for packets with vulnerable IVs
 - Look for “weak IVs”
 - Collect the corresponding keystream bytes
- **Step 3:** Apply statistical analysis
 - Use FMS attack to determine most likely key byte values
 - Incrementally build up the secret key
- **Step 4:** Verify the recovered key
 - Test against captured packets
 - Success: Complete compromise of the Wi-Fi network!

Attacks only get better

- **Klein's Attack (2005):** Extended the vulnerable IV patterns
 - Found additional weak key structures
 - Reduced the number of packets needed for key recovery
- **PTW Attack (2007):** Pyshkin, Tews, and Weinmann
 - Revolutionary approach using Bayes' theorem
 - Could crack WEP with as few as 40,000 packets
 - Made attacks feasible on even low-traffic networks
- **Key Recovery Speedup:** From hours to minutes
 - Modern attacks can crack WEP in under 10 minutes
 - Some demonstrations crack it in under 60 seconds

Lessons learned from WEP

- **Lesson 1:** Never use a stream cipher with related keys
 - WEP's IV structure created exactly this scenario
 - Modern protocols use proper key derivation functions
- **Lesson 2:** Academic attacks become practical faster than expected
 - FMS attack went from paper to tool in months
 - "Theoretical" often means "practical soon"
- **Lesson 3:** Security protocols need formal analysis
 - WEP was designed by committee without cryptographic rigor
 - Modern protocols undergo extensive peer review
- **Lesson 4:** Backward compatibility can be a security nightmare
 - WEP remained deployed long after being broken
 - Organizations were slow to migrate due to cost/complexity

WEP timeline

- **2001:** FMS attack published - WEP theoretically broken
- **2001:** AirSnort released - WEP practically broken
- **2003:** WPA (Wi-Fi Protected Access) standardized as replacement
- **2004:** WPA2 released with AES encryption
- **2006:** Wi-Fi Alliance deprecates WEP
- **2012:** WEP support removed from Windows 8
- **2018:** Many vendors stop supporting WEP entirely
- **Today:** WEP is considered a historical curiosity

Section 3

RC4 in TLS

RC4 in TLS: from preferred to prohibited

- **RC4's role in TLS/SSL:** Once the default stream cipher
 - Preferred over block ciphers due to speed
 - No need for padding (unlike CBC mode)
 - Avoided the BEAST attack against CBC ciphers
- **2011:** BEAST attack made RC4 more attractive
 - Attack on CBC mode in TLS 1.0
 - Many servers switched to RC4-only cipher suites
 - RC4 usage actually increased!
- **The irony:** Avoiding one attack led directly into another
 - RC4's biases were known but considered impractical
 - This assumption would prove catastrophically wrong

AlFardan et al. (2013): RC4 biases in TLS

- **The discovery:** RC4's biases are exploitable in TLS:^a
 - First 256 bytes of keystream heavily biased
 - Certain byte positions more predictable than others
 - Biases persist even with proper key generation
- **Key finding:** Single-byte biases
 - Position 1: $\Pr[Z_1 = 0] \approx 2^{-7}$ (double expected)
 - Position 2: $\Pr[Z_2 = 0] \approx 2^{-7}$ (double expected)
 - Positions 3-255: Various smaller biases
- **Multi-byte biases:** Even stronger patterns
 - $(Z_1, Z_{257}) = (0, 0)$ with probability 2^{-11}
 - Should be 2^{-16} if truly random
 - 32 times more likely than expected!

^a<https://appliedcryptography.page/paper/#rc4-tls>

The broadcast attack scenario

- **Attack model:** Same plaintext encrypted many times
 - HTTP cookies sent with every request
 - Session tokens in fixed positions
 - Password fields in web forms
- **The mathematics:** Bias accumulation
 - Collect ciphertexts: $C_1, C_2, \dots, C_n$
 - For position i : count occurrences of each byte value
 - Most frequent ciphertext byte reveals plaintext byte
 - Required samples: $\mathcal{O}(N \cdot B^{-2})$ where B is bias
- **Practical requirements:**
 - 2^{24} to 2^{30} encryptions needed
 - Feasible for persistent attackers
 - JavaScript injection can generate traffic

Garman et al. (2015): attacks only get better

- **The challenge:** AlFardan et al.'s attack was still “impractical”
 - Required billions of encryptions
 - Took days or weeks to execute
 - Many dismissed it as theoretical
- **Insight:** Target password verifiers, not cookies^a
 - Basic Authentication sends passwords in every request
 - IMAP/SMTP use similar repeated authentication
 - Password structure provides additional constraints
- **The improvements:** Orders of magnitude faster
 - Exploit password character distributions
 - Use Mantin's ABSAB bias (positions 1-4)
 - Combine with dictionary attacks
 - Other attack papers use similar techniques, including to break WPA-TKIP, a successor to WEP!^a

^a<https://appliedcryptography.page/paper/#rc4-attacks>

^a<https://appliedcryptography.page/paper/#rc4-biases>

Mantin's ABSAB bias (2005)

- **The discovery:** Certain digraph patterns repeat with anomalous frequency^a
 - Pattern: Two characters repeat after a gap (e.g., ABAB, ABCAB)
 - Occurs when value 1 is used to update index j in RC4
 - Creates correlations between distant output positions
- **The mathematics:** For pattern AB...AB with gap G
 - Expected probability in random: $1/N^2$
 - Actual probability in RC4: $(1 + e^{(-4-8G)/N}/N) \cdot 1/N^2$
 - Relative bias: $\approx 1/N$ for zero gap, decreases with gap size
- **Why it matters:**
 - Enables distinguishing attacks with only 2^{26} samples
 - Can predict individual bytes with 82% accuracy after 2^{50} samples
 - Particularly effective against password-based authentication

^a<https://appliedcryptography.page/paper/#rc4-absab>

ABSAB bias in action: practical example

- **Target:** Password “hello123” in HTTP Basic Authentication
- **Step 1:** Password encoded as base64: “aGVsbG8xMjM=”
- **Step 2:** Look for ABSAB patterns in collected ciphertexts:



↑ Repeated “ll” pattern

ABSAB: (l, l, *, l, l) occurs 1024× more!

- **Step 3:** After 2^{26} samples, frequency analysis reveals:
 - Positions 2-3: “ll” pattern confirmed via ABSAB
 - Position 0: “h” (68/256 occurrences)
 - Position 1: “e” (65/256 occurrences)
 - Password structure revealed: “h_ll__”
- **Result:** Search space reduced from 62^8 to 62^4 possibilities!

Password recovery: the numbers

- **Basic Authentication attack performance:**
 - 6-character passwords: 2^{26} encryptions
 - Time: ≈ 50 –100 hours.
- **IMAP attack results:**
 - Real passwords recovered in laboratory setting
 - Character-by-character recovery possible

POODLE (2014): the final nail

- **POODLE:** Padding Oracle On Downgraded Legacy Encryption
 - Not specifically an RC4 attack
 - But exploited SSL 3.0's MAC-then-encrypt with CBC
 - Led to mass exodus from SSL 3.0
- **The downgrade dance:**
 - Attackers force TLS connections to fail
 - Browsers retry with older SSL 3.0
 - SSL 3.0 has no downgrade protection!
- **Why this matters for RC4:**
 - SSL 3.0 heavily used RC4
 - POODLE + RC4 biases = from compromise to compromise
 - Final push to deprecate both SSL 3.0 and RC4

Coordinated response between vendors

- **2013:** IETF begins discussing RC4 deprecation
 - Working groups start planning migration
- **2014/2015:** Major browsers act
 - Chrome/Firefox announce RC4 phase-out plans
 - Microsoft issues security advisories
 - Apple begins removing RC4 from Safari
- **2015:** RFC 7465 prohibits RC4 in TLS
 - “TLS clients MUST NOT include RC4 cipher suites”
 - “TLS servers MUST NOT select RC4 cipher suites”
 - Formal end of RC4 in internet protocols
- **2016/2018:** RC4 removed from all major browsers

Technical lessons from RC4's demise

- **Lesson 1:** Statistical biases matter at scale
 - “Small” biases become exploitable with enough data
 - Modern internet generates massive data volumes
 - Never dismiss statistical irregularities
- **Lesson 2:** Attacks combine in unexpected ways
 - ABSAB bias + password structure = practical attack
 - Protocol downgrade + cipher weakness = disaster
- **Lesson 3:** Migration is painful but necessary
 - RC4 removal broke many legacy systems
 - But waiting would have been catastrophic
 - Plan for crypto-agility from the start

RC4: a complete timeline

- **1987:** Ron Rivest designs RC4
- **1994:** Algorithm leaked to cypherpunks
- **1995-2000:** Early bias discoveries
- **2001:** FMS attack breaks WEP
- **2011:** BEAST attack increases RC4 usage
- **2013:** AlFardan et al. show TLS attacks are almost practical
- **2014:** POODLE compromises SSL 3.0
- **2015:** Garman et al. improve password recovery
- **2015:** IETF prohibits RC4 (RFC 7465)
- **2016:** RC4 removed from browsers
- **Today:** RC4 is cryptographically dead

The broader impact

- **On cryptographic design:**
 - Stream ciphers now use different designs (ChaCha20)
 - Focus on provable security properties
 - Extensive bias analysis before deployment
- **On protocol design:**
 - TLS 1.3 removed all weak ciphers
 - Mandatory authenticated encryption (AEAD)
 - No more downgrade attacks
- **On deployment practices:**
 - Regular security audits of cipher suites
 - Sunset dates for algorithms
 - Crypto-agility built into protocols

Modern alternatives to RC4

- **ChaCha20-Poly1305:** The modern stream cipher
 - Designed by Daniel J. Bernstein
 - No known biases after extensive analysis
 - Includes authentication (AEAD)
- **AES-GCM:** The block cipher alternative
 - Hardware acceleration on modern CPUs
 - Well-studied security properties
 - Standard choice for TLS 1.3
- **Key differences from RC4:**
 - Authenticated encryption prevents tampering
 - No statistical biases in keystream
 - Designed with modern cryptanalysis in mind

RC4: final thoughts

- **A cipher of its time:**
 - Perfect for 1990s hardware constraints
 - Served the early internet well
 - But couldn't survive modern cryptanalysis
- **The lifecycle of cryptography:**
 - All ciphers eventually fall
 - Plan for migration from day one
 - Monitor academic literature for early warnings
- **The human element:**
 - Ron Rivest's elegant design lasted 28 years
 - Cryptographers who broke it built on decades of research
 - The cycle continues with new algorithms and new attacks



Applied Cryptography

Summer 2026

Part 2: Real-World Cryptography

2.2: The Story of RC4

Nadim Kobeissi

<https://appliedcryptography.page>